Unit V – Virtual Functions, Polymorphism & Exception Handling: Answers

Corrected code only

Section A – Debugging Questions

Question 1: Non-Virtual Destructor Leak

```
#include <iostream>
class Shape {
public:
    Shape() { std::cout << "Shape created." << std::endl; }
    virtual ~Shape() { std::cout << "Shape destroyed." << std::endl; }
    virtual void render() const { std::cout << "Rendering generic shape." << std::endl; }
};
class ComplexPolygon : public Shape {
private:
    double *vertexArray;
    int vertexCount;
public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        std::cout << "Allocated " << vertexCount << " vertices on heap." << std::endl;
    }
    ~ComplexPolygon() override {
        std::cout << "Deallocating polygon vertex heap memory." << std::endl;
        delete[] vertexArray;
    }
    void render() const override {
        std::cout << "Rendering polygon with " << vertexCount << " vertices." << std::endl;
    }
};
int main() {
    Shape *shapePtr = new ComplexPolygon(1000);
    shapePtr->render();
    delete shapePtr;
    return 0;
}
```

Question 2: Inverted Catch Handler Order

```
#include <iostream>
#include <string>
class BankingException {
protected:
    std::string msg;
public:
    BankingException(std::string m) : msg(m) {}
    virtual ~BankingException() {}
    virtual std::string getMessage() const { return msg; }
};
class InsufficientFundsException : public BankingException {
private:
    double shortfall;
public:
    InsufficientFundsException(std::string m, double sf) : BankingException(m), shortfall(sf) {}
    double getShortfall() const { return shortfall; }
};
void processWithdrawal(double balance, double amount) {
    if (amount > balance)
        throw InsufficientFundsException("Account overdrawn!", amount - balance);
}
```

```
int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    catch (const InsufficientFundsException &ife) {      // most specific first
        std::cout << ife.getMessage() << " Shortfall of: $" << ife.getShortfall() << std::endl;
    }
    catch (const BankingException &be) {
        std::cout << "General Banking Error: " << be.getMessage() << std::endl;
    }
    catch (...) {
        std::cout << "Unknown error" << std::endl;
    }
    return 0;
}
```

Question 3: throw e; vs throw;

```
#include <iostream>
#include <exception>
#include <string>
class DatabaseError : public std::exception {
protected:
    std::string errorDetails;
public:
    DatabaseError(std::string msg) : errorDetails(msg) {}
    const char* what() const noexcept override { return errorDetails.c_str(); }
};
class TimeoutError : public DatabaseError {
public:
    int timeoutMs;
    TimeoutError(std::string msg, int ms) : DatabaseError(msg), timeoutMs(ms) {}
};
void executeQuery() { throw TimeoutError("Query timed out connecting to shard 3", 5000); }
void intermediateAuditor() {
    try { executeQuery(); }
    catch (const DatabaseError &e) {
        std::cout << "[AUDIT LOG]: Intercepted DB error." << std::endl;
        throw;                                  // rethrow the ORIGINAL object
    }
}
int main() {
    try { intermediateAuditor(); }
    catch (const TimeoutError &te) {
        std::cout << "Client caught TimeoutError! Duration: " << te.timeoutMs << " ms" <<
std::endl;
    }
    catch (const DatabaseError &de) {
        std::cout << "Client caught generic DatabaseError: " << de.what() << std::endl;
    }
    return 0;
}
```

Question 4: Instantiating an Abstract Class

```
#include <iostream>
class NetworkPacket {
public:
    virtual ~NetworkPacket() {}
    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};
class VideoPacket : public NetworkPacket {
private:
    int frameID;
public:
```

```
    VideoPacket(int id) : frameID(id) {}
    void serialize() override {
        std::cout << "Serializing H.264 Video Frame: " << frameID << std::endl;
    }
    int getPacketSize() override { return 1500; }
};
int main() {
    VideoPacket packet(1042);
    packet.serialize();
    std::cout << "Packet size: " << packet.getPacketSize() << " bytes" << std::endl;
    NetworkPacket *p = &packet;
    p->serialize();
    return 0;
}
```

Question 5: Pass-by-Value Slicing

```
#include <iostream>
class Entity {
public:
    virtual ~Entity() {}
    virtual void attack() const { std::cout << "Entity performs generic scratch." << std::endl; }
};
class Dragon : public Entity {
public:
    void attack() const override { std::cout << "Dragon breathes scorching fire!" << std::endl; }
};
void triggerEncounter(const Entity &e) { e.attack(); }
int main() {
    Dragon boss;
    triggerEncounter(boss);
    return 0;
}
```

Section B – Pseudo-Code Questions

Question 1: Universal Payment Gateway

```
CLASS PaymentGateway ABSTRACT
  PUBLIC:
    ABSTRACT METHOD Authorize(REAL amount) RETURNS BOOLEAN
    ABSTRACT METHOD Capture() RETURNS BOOLEAN
    VIRTUAL DESTRUCTOR ~PaymentGateway()
ENDCLASS

CLASS CreditCardGateway INHERITS PaymentGateway
  PRIVATE:
    STRING cardNumber, cvv
    REAL authorizedAmount
  PUBLIC:
    CONSTRUCTOR CreditCardGateway(STRING card, STRING code)
      SET cardNumber = card
      SET cvv = code
      SET authorizedAmount = 0
    ENDCONSTRUCTOR
    METHOD Authorize(REAL amount) RETURNS BOOLEAN OVERRIDE
      IF LENGTH(cardNumber) = 16 AND LENGTH(cvv) = 3 AND amount > 0 THEN
        SET authorizedAmount = amount
        RETURN TRUE
      ENDIF
      RETURN FALSE
    ENDMETHOD
    METHOD Capture() RETURNS BOOLEAN OVERRIDE
      IF authorizedAmount > 0 THEN
```

```
        DISPLAY "Card charged: $", authorizedAmount
        SET authorizedAmount = 0
        RETURN TRUE
      ENDIF
      RETURN FALSE
    ENDMETHOD
ENDCLASS

CLASS CryptoGateway INHERITS PaymentGateway
  PRIVATE:
    STRING walletHash
    REAL gasFee, balance
    REAL authorizedAmount
  PUBLIC:
    CONSTRUCTOR CryptoGateway(STRING wallet, REAL fee, REAL walletBalance)
      SET walletHash = wallet
      SET gasFee = fee
      SET balance = walletBalance
      SET authorizedAmount = 0
    ENDCONSTRUCTOR
    METHOD Authorize(REAL amount) RETURNS BOOLEAN OVERRIDE
      IF SUBSTRING(walletHash, 0, 2) = "0x" AND amount > 0 AND balance >= amount + gasFee THEN
        SET authorizedAmount = amount
        RETURN TRUE
      ENDIF
      RETURN FALSE
    ENDMETHOD
    METHOD Capture() RETURNS BOOLEAN OVERRIDE
      IF authorizedAmount > 0 THEN
        SET balance = balance - authorizedAmount - gasFee
        DISPLAY "Crypto transferred: $", authorizedAmount, " (gas $", gasFee, ")"
        SET authorizedAmount = 0
        RETURN TRUE
      ENDIF
      RETURN FALSE
    ENDMETHOD
ENDCLASS

BEGIN Main
  INTEGER choice
  POINTER TO PaymentGateway gatewayPtr
  READ choice                                    // 1 = card, 2 = crypto
  IF choice = 1 THEN
    SET gatewayPtr = NEW CreditCardGateway("4111111111111111", "123")
  ELSE
    SET gatewayPtr = NEW CryptoGateway("0xA1B2C3", 12.50, 5000.00)
  ENDIF
  IF gatewayPtr->Authorize(1500.00) = TRUE THEN      // late binding
    IF gatewayPtr->Capture() = TRUE THEN
      DISPLAY "Payment complete"
    ENDIF
  ELSE
    DISPLAY "Authorization declined"
  ENDIF
  DELETE gatewayPtr                                // virtual destructor
END
```

Question 2: Resilient Banking Engine – Nested Try-Catch & Rethrow

```
CLASS TransactionException
  PROTECTED:
    STRING reason
  PUBLIC:
    CONSTRUCTOR TransactionException(STRING r)
      SET reason = r
    ENDCONSTRUCTOR
    VIRTUAL METHOD GetReason() RETURNS STRING
```

```
      RETURN reason
    ENDMETHOD
ENDCLASS

CLASS AccountFrozenException INHERITS TransactionException
  PUBLIC:
    CONSTRUCTOR AccountFrozenException() : TransactionException("Account is frozen")
    ENDCONSTRUCTOR
ENDCLASS

CLASS DailyLimitExceededException INHERITS TransactionException
  PRIVATE:
    REAL excessAmount
  PUBLIC:
    CONSTRUCTOR DailyLimitExceededException(REAL excess)
        : TransactionException("Daily limit exceeded")
      SET excessAmount = excess
    ENDCONSTRUCTOR
    METHOD GetExcess() RETURNS REAL
      RETURN excessAmount
    ENDMETHOD
ENDCLASS

CONSTANT REAL DAILY_LIMIT = 50000.0

FUNCTION DebitLedger(BOOLEAN isFrozen, REAL todayTotal, REAL amount)
  IF isFrozen = TRUE THEN
    THROW AccountFrozenException()
  ENDIF
  IF todayTotal + amount > DAILY_LIMIT THEN
    THROW DailyLimitExceededException(todayTotal + amount - DAILY_LIMIT)
  ENDIF
  DISPLAY "Ledger debited: $", amount
ENDFUNCTION

FUNCTION AuditTransactionWrapper(BOOLEAN isFrozen, REAL todayTotal, REAL amount)
  TRY
    DebitLedger(isFrozen, todayTotal, amount)
  CATCH (REF TransactionException ex)
    DISPLAY "[SECURITY AUDIT] Failed transaction: ", ex.GetReason()
    RETHROW                                    // same object, same dynamic type
  ENDTRY
ENDFUNCTION

BEGIN Main
  TRY
    AuditTransactionWrapper(TRUE, 0.0, 1000.0)
    DISPLAY "Transaction successful"
  CATCH (REF AccountFrozenException ex)
    DISPLAY "Client notice: your account is frozen. Contact the bank."
  CATCH (REF DailyLimitExceededException ex)
    DISPLAY "Client notice: daily limit exceeded by $", ex.GetExcess()
  CATCH (REF TransactionException ex)
    DISPLAY "Client notice: transaction failed - ", ex.GetReason()
  CATCH (...)
    DISPLAY "Unexpected system error"
  ENDTRY
END
```

Question 3: Polymorphic Game Engine with Virtual Destructors

```
CLASS GameObject ABSTRACT
  PUBLIC:
    ABSTRACT METHOD Update(REAL deltaTime)
    ABSTRACT METHOD Render()
    VIRTUAL DESTRUCTOR ~GameObject()
      DISPLAY "GameObject destroyed"
```

```
    ENDDESTRUCTOR
ENDCLASS

CLASS ParticleSystem INHERITS GameObject
  PRIVATE:
    POINTER TO REAL particleX
    INTEGER count
  PUBLIC:
    CONSTRUCTOR ParticleSystem(INTEGER n)
      SET count = n
      SET particleX = ALLOCATE ARRAY OF REAL [count]
      INTEGER i
      FOR i = 0 TO count - 1 STEP 1 DO
        SET particleX[i] = 0.0
      ENDFOR
    ENDCONSTRUCTOR
    METHOD Update(REAL deltaTime) OVERRIDE
      INTEGER i
      FOR i = 0 TO count - 1 STEP 1 DO
        SET particleX[i] = particleX[i] + (deltaTime * 10.0)
      ENDFOR
    ENDMETHOD
    METHOD Render() OVERRIDE
      DISPLAY "Rendering ", count, " particles"
    ENDMETHOD
    DESTRUCTOR ~ParticleSystem() OVERRIDE
      DISPLAY "Releasing particle array"
      DEALLOCATE particleX
    ENDDESTRUCTOR
ENDCLASS

BEGIN Main
  ARRAY sceneEntities[2] OF POINTER TO GameObject
  INTEGER i
  SET sceneEntities[0] = NEW ParticleSystem(100)
  SET sceneEntities[1] = NEW ParticleSystem(50)
  FOR i = 0 TO 1 STEP 1 DO
    sceneEntities[i]->Update(0.016)
    sceneEntities[i]->Render()
  ENDFOR
  FOR i = 0 TO 1 STEP 1 DO
    DEALLOCATE sceneEntities[i]              // virtual destructor chain
  ENDFOR
END
```

Question 4: File Pipeline – RAII & Stack Unwinding

```
CLASS ResourceGuard
  PRIVATE:
    INTEGER handle
    STRING name
  PUBLIC:
    CONSTRUCTOR ResourceGuard(STRING fileName)
      SET name = fileName
      SET handle = OpenHandle(fileName)
      DISPLAY "Acquired handle for ", name
    ENDCONSTRUCTOR
    DESTRUCTOR ~ResourceGuard()
      CloseHandle(handle)
      DISPLAY "Released handle for ", name
    ENDDESTRUCTOR
    METHOD ReadBlock() RETURNS INTEGER
      RETURN ReadNextBlock(handle)
    ENDMETHOD
ENDCLASS

CLASS CRCValidationException
```

```
  PUBLIC:
    INTEGER expectedCRC, calculatedCRC
    CONSTRUCTOR CRCValidationException(INTEGER e, INTEGER c)
      SET expectedCRC = e
      SET calculatedCRC = c
    ENDCONSTRUCTOR
ENDCLASS

FUNCTION IngestDataStream(STRING fileName)
  ResourceGuard guard(fileName)              // local automatic object
  INTEGER expected, calculated
  SET expected = ReadHeaderCRC(guard)
  SET calculated = ComputeCRC(guard)
  IF calculated NOT= expected THEN
    THROW CRCValidationException(expected, calculated)
  ENDIF
  DISPLAY "Ingestion succeeded"
ENDFUNCTION                                   // guard destroyed on normal exit too

BEGIN Main
  TRY
    IngestDataStream("flight.dat")
  CATCH (REF CRCValidationException ex)
    DISPLAY "CRC error: expected ", ex.expectedCRC, " calculated ", ex.calculatedCRC
  CATCH (...)
    DISPLAY "Unknown failure during ingestion"
  ENDTRY
  DISPLAY "Pipeline finished"
END
```

Question 5: Telemetry Calibration with noexcept

```
CLASS OutOfBoundsException
  PUBLIC:
    REAL badValue
    CONSTRUCTOR OutOfBoundsException(REAL v)
      SET badValue = v
    ENDCONSTRUCTOR
ENDCLASS

CONSTANT REAL MIN_TEMP = -273.15
CONSTANT REAL MAX_TEMP = 1500.0

FUNCTION CalibrateThermalSensors(REAL rawVoltage) RETURNS REAL NOEXCEPT(FALSE)
  REAL temp
  SET temp = (rawVoltage * 100.0) - 50.0
  IF temp < MIN_TEMP OR temp > MAX_TEMP THEN
    THROW OutOfBoundsException(temp)
  ENDIF
  RETURN temp
ENDFUNCTION

FUNCTION CommitTelemetry(REAL finalTemp) NOEXCEPT
  TRY
    WriteToAvionicsBus(finalTemp)             // may fail internally
  CATCH (...)
    LogLocalFault("Bus write failed, reading suppressed")   // swallow: never escapes
  ENDTRY
ENDFUNCTION

BEGIN Main
  REAL raw, temp
  READ raw
  TRY
    SET temp = CalibrateThermalSensors(raw)
    CommitTelemetry(temp)                     // never throws
    DISPLAY "Telemetry committed: ", temp, " C"
```

```
  CATCH (REF OutOfBoundsException ex)
    DISPLAY "Calibration rejected: out-of-range temperature ", ex.badValue, " C"
  CATCH (...)
    DISPLAY "Unexpected calibration failure"
  ENDTRY
END
```